VACCINE COLD CHAIN MONITOR · FIRMWARE V2

Built-in animated dashboard, hosted on the ESP32 with LittleFS

What was added to your project, how it works, and a step-by-step guide to building it in hardware: wiring, uploading, using, testing and fixing problems.



PROJECT FOLDER
~/Downloads/coldchain_project_littlefs/

BOARD
ESP32 Dev Module (ESP32-WROOM-32)

PREPARED
23 September 2026

Contents

1. What was done
2. How the system works
3. Parts list
4. Wiring the hardware
5. Software setup and upload
6. Opening and using the dashboard
7. Testing checklist
8. Troubleshooting
9. Files and what changed
10. What was verified, and what still needs testing

1. What was done

Your original project (`coldchain_project.zip`) had an ESP32 that reads the sensors, sends readings to a Flask server on a laptop, and saves them to an SD card when WiFi is down. The dashboard was only on the laptop, so if the laptop or router was off, **nobody could see the temperature except on the tiny OLED**.

Version 2 adds a **website that is stored inside the ESP32's own flash memory (LittleFS)** and served by the ESP32 itself. Any phone or laptop can open it, with or without a router, and without internet.

Feature	Before (v1)	Now (v2)
See live readings on a phone	**Only via laptop + Flask**	**Directly from the ESP32, any browser**
Works with no router / no internet	**No (OLED only)**	**Yes, the unit makes its own WiFi hotspot**
Animated gauge, thermometer, humidity wave, charts	**—**	**Yes, smooth animations, phone-friendly**
Dashboard refresh	Every 30 s reading	**Sensors sampled every 5 s for the live view**
History after a reboot	**Lost from the screen**	**Last 2 h reloaded from the on-board log**
Silence the buzzer	**Pull the power**	**"Silence alarm" button (10 min; the LED keeps flashing)**
Download data without a PC server	**Remove the SD card**	**CSV download buttons on the page**
ML holdover prediction	Flask dashboard only	**Also shown on the unit when the server is reachable**
Clock accuracy	RTC drifts	**RTC corrected from internet time whenever online**
Flask upload + SD backlog + sync	Yes	**Unchanged (same API, same CSV format)**

What's on the dashboard

Status banner

Big SAFE / TOO WARM / TOO COLD message with icon, how long the excursion has lasted, and what to do.

Product gauge

DS18B20 core temperature on an animated dial with the 2–8 °C zone shaded. Shows the rate of change per minute.

Box air thermometer

DHT22 air temperature as a rising mercury column.

Humidity wave

Animated liquid level showing relative humidity, with a condensation warning above 70%.

Holdover hourglass

Hours of safe storage left, predicted by your RandomForest model on the Flask server.

History charts

Live 10 min or 2 h trend. Safe band and excursions shaded. Hover or tap for exact values. Table view.

Time-in-range ring

% of logged readings inside 2–8 °C and the number of excursion events.

System health

WiFi signal, hotspot, Flask server, SD backlog rows, flash usage, uptime.

Event log + downloads

Timeline of alarms, WiFi drops and syncs, plus CSV download buttons.

2. How the system works

Think of the ESP32 as three things at once: a **data logger**, a **small web server**, and a **WiFi hotspot**.

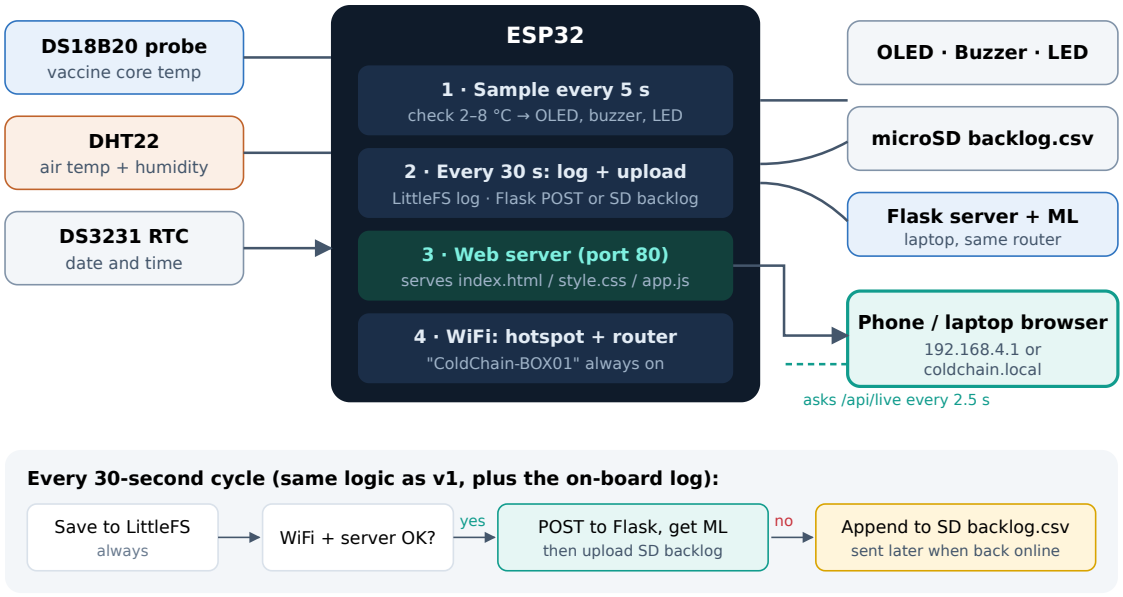


Figure 1. Data flow. The dashboard only talks to the ESP32. The Flask server is optional: it adds long-term storage and the ML prediction.

Why LittleFS?

The ESP32 has 4 MB of flash. Most of it holds the program, and a separate area can hold files like a tiny USB drive. **LittleFS** is the file system for that area. It is designed to survive sudden power loss, which matters in a cold box running on a power bank. It holds two things:

- **The website**: `index.html`, `style.css`, `app.js` (66 KB in total, no internet libraries needed).
- **An on-board reading log**: `/log/readings.csv`, one row every 30 s. When it reaches about 96 KB (roughly 1.5 days), it is renamed to `readings_old.csv` and a new file starts, so flash never fills up (about the last 3 days stay on the unit). The SD card is still the separate *upload queue* for Flask.

3. Parts list

#	Part	Qty	Notes
1	**ESP32 Dev board** (ESP32-WROOM-32, 30 or 38 pin)	1	Must have 4 MB flash (almost all do). Not ESP32-S2/C3: pin numbers differ.
2	**DS18B20 waterproof probe** (steel tip, 1 m cable)	1	Put it inside a small bottle of water/glycol to act like a vaccine vial.
3	**4.7 kΩ resistor**	1	Pull-up for the DS18B20 data line. **Required**, or you will read −127 °C.
4	**DHT22 (AM2302) module**	1	A 3-pin module has its pull-up built in. For the bare 4-pin sensor, add 10 kΩ from DATA to 3V3.
5	**DS3231 RTC module** + CR2032 cell	1	Keeps time without WiFi.
6	**SSD1306 OLED 0.96" I2C** (128×64, address 0x3C)	1	4-pin I2C version (VCC, GND, SCL, SDA).
7	**MicroSD card module** (SPI) + 4–32 GB card	1	Format the card as **FAT32**.
8	**Active buzzer** (5 V/3.3 V)	1	An "active" buzzer beeps by itself when powered.
9	**Red LED** + **220 Ω resistor**	1 each	Excursion indicator.
10	Breadboard / perfboard, jumper wires	—	Perfboard + solder is recommended for the final unit (vibration in transport).
11	USB cable + 5 V power bank (≥ 10 000 mAh)	1	The ESP32 with WiFi uses about 0.1–0.25 A, so 10 000 mAh lasts roughly 1–2 days.
12	Cold box / vaccine carrier, small plastic enclosure	1	Keep the electronics **outside** the cold compartment (see below).

Physical layout tips

- **Electronics outside, probe inside.** Mount the ESP32, OLED and SD in a small box on the *outside* of the cold box lid. Cold electronics collect condensation when they warm up.
- **Only the DS18B20 cable goes in.** Pass it through the lid seal. It is thin enough that the lid still closes.
- **DHT22 placement:** inside the cold box, hanging in the air and not touching ice packs, if you want box-air temperature. Use a 4-core cable up to ~2 m.
- **Probe in a "dummy vial":** tip submerged in a 10–30 ml bottle of water or 10% glycol, placed among the vaccines. This reads what the *product* feels, not the air.
- **OLED visible** through a window in the enclosure. It shows the dashboard address.
- **Buzzer hole** in the enclosure so the alarm is audible.

4. Wiring the hardware

The pin map is **unchanged from v1**. If your v1 unit is already wired, you don't need to rewire anything.

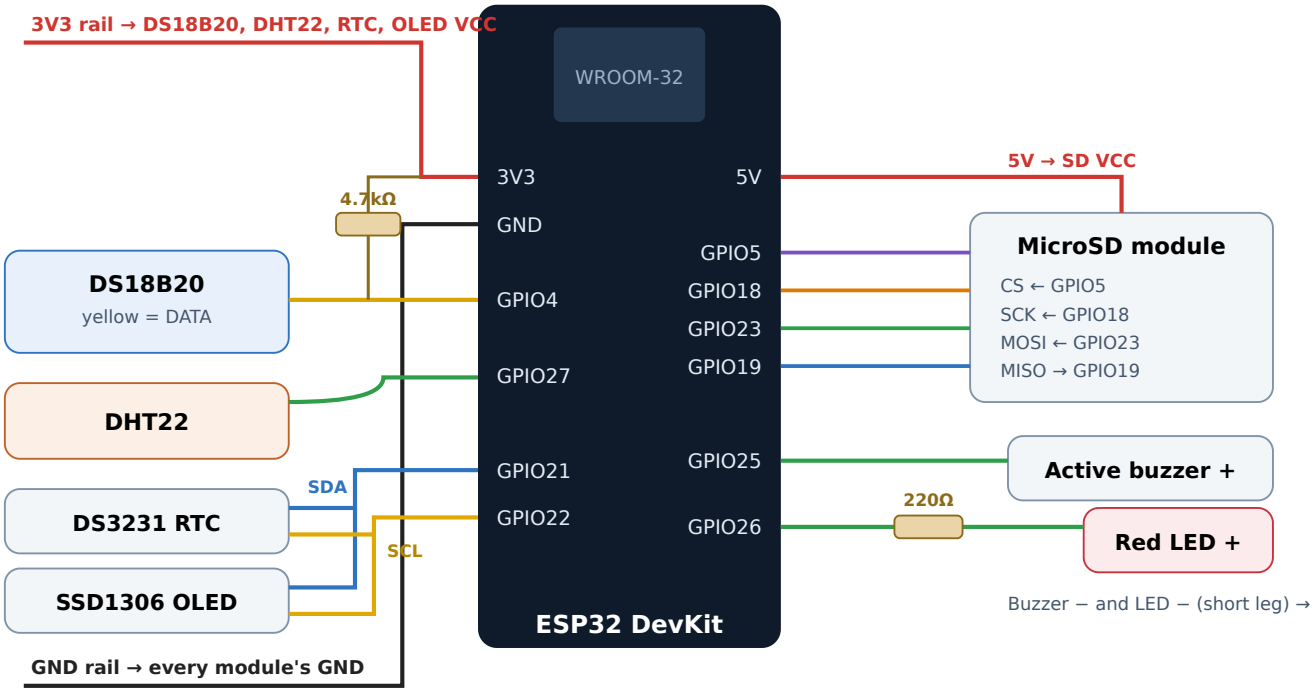


Figure 2. Wiring overview. Every module also connects to 3V3 (red) and GND (black); the SD module takes 5V.

Module	Module pin	ESP32 pin	Tip
DS18B20	Red (VCC)	`3V3`	**4.7 kΩ between yellow (DATA) and 3V3.** Without it the reading is −127 °C.
	Black (GND)	`GND`	
	Yellow (DATA)	`GPIO 4`	
DHT22	+ / VCC, − / GND	`3V3, GND`	Bare 4-pin sensor: pin 3 not used; add 10 kΩ DATA→3V3.
	OUT / DATA	`GPIO 27`	
DS3231 RTC	VCC, GND	`3V3, GND`	RTC and OLED **share** the same SDA/SCL wires. That is normal: they have different I2C addresses (0x68, 0x3C).
	SDA, SCL	`GPIO 21, GPIO 22`	
SSD1306 OLED	VCC, GND	`3V3, GND`	
	SDA, SCL	`GPIO 21, GPIO 22`	
MicroSD	VCC, GND	`5V (VIN), GND`	Most blue SD modules have a 3.3 V regulator and need 5 V. A module marked "3.3V only" goes to 3V3.
	CS, SCK	`GPIO 5, GPIO 18`	
	MOSI, MISO	`GPIO 23, GPIO 19`	

Module	Module pin	ESP32 pin	Tip
Buzzer	+ / −	`GPIO 25 / GND`	Active buzzer. Check polarity (longer leg is +).
LED	Anode (long) via 220 Ω / cathode	`GPIO 26 / GND`	The resistor can be on either leg.

Before powering on

Check with a multimeter that 3V3 and GND are not shorted, and that nothing on the 3V3 rail is connected to 5V. A 5 V signal on an ESP32 GPIO can damage it.

5. Software setup and upload

1 **Install Arduino IDE 2 and the ESP32 board package**

File → Preferences → "Additional boards manager URLs", then paste: `https://raw.githubusercontent.com/espressif/arduino-esp32/gh-pages/package_esp32_index.json`

Tools → Board → Boards Manager → search **esp32** → install **esp32 by Espressif Systems** (version 3.x; tested with 3.3.10).

2 **Install the libraries**

(Sketch → Include Library → Manage Libraries)

DHT sensor library + **Adafruit Unified Sensor** · **OneWire** · **DallasTemperature** · **RTClib** · **Adafruit SSD1306** + **Adafruit GFX Library** · **ArduinoJson** (**version 7.x**).

LittleFS, WebServer, ESPmDNS, SD, WiFi and HTTPClient come with the ESP32 package, so there's nothing extra to install.

3 **Open the sketch**

`firmware/cold_chain_esp32/cold_chain_esp32.ino` . The `data` folder next to it holds the website. Keep that folder name and location exactly as they are.

4 **Choose board settings**

Tools → Board → **ESP32 Dev Module**, then Tools → Port → your COM port, then Tools → Partition Scheme → **No OTA (2MB APP/2MB SPIFFS)**.

> **Important**
> With the default scheme the program uses 96% of its space. With "No OTA" it uses 60%, and the website/log get about 1.9 MB. You must choose the same scheme for both uploads (steps 6 and 7).

5 **Edit the USER CONFIG block**

at the top of the sketch:

```
const char* WIFI_SSID     = "YourRouterName";     // optional; the hotspot works
without it
const char* WIFI_PASSWORD = "YourRouterPassword";
const char* AP_SSID       = "ColdChain-BOX01";    // the unit's own hotspot
const char* AP_PASSWORD   = "coldchain123";       // CHANGE this (min 8 characters)
const char* SERVER_HOST   = "192.168.1.100";      // laptop IP running Flask (optional)
const long  GMT_OFFSET_SEC = 19800;               // India = UTC+5:30
```

6 Upload the website to LittleFS (one time)

Install the plugin: download `arduino-littlefs-upload-x.x.x.vsix` from `github.com/earlephilhower/arduino-littlefs-upload/releases` and copy it into `C:\Users\<you>\.arduinoIDE\plugins\` (Linux/Mac: `~/.arduinoIDE/plugins/`). Create the folder if it doesn't exist, then restart the IDE.

Close the Serial Monitor, then press **Ctrl + Shift + P**, type **Upload LittleFS to Pico/ESP8266/ESP32** and press Enter. Wait for "Completed upload".

7 Upload the program

Click the normal **Upload (→)** button. If it's stuck at "Connecting.....", hold the **BOOT** button on the ESP32 until it starts writing.

8 Check the Serial Monitor (115200 baud)

You should see:

```
[FS] LittleFS mounted. Used 81920 / 1966080 bytes
[SD] Card mounted OK.
[AP] Hotspot 'ColdChain-BOX01' up. Dashboard: http://192.168.4.1
[WEB] Dashboard server started on port 80.
[EVENT] Unit booted - monitoring started
[WiFi] Connected. IP: 192.168.1.57        (only if the router details are set)
```

The byte counts on your board will differ slightly.

Changing the website later

Edit files in `data/` and repeat step 6 only. Uploading the program again does **not** erase the website or the log. The browser caches files for 5 minutes, so do a hard refresh (Ctrl+F5) to see changes.

Preview without hardware

Run `python3 tools/mock_device.py` in the project folder and open `http://localhost:8080`. It serves the same website with **simulated** data (marked "SIMULATED" on the page), which is useful for a demo or viva.

6. Opening and using the dashboard

Where you are	On your phone/laptop	Type in the browser
In the field / no router	Connect to WiFi **ColdChain-BOX01** (password as set). If the phone says "No internet", choose **Stay connected**.	`http://192.168.4.1`
Lab / same router as the unit	Stay on your normal WiFi	`http://coldchain.local` or the IP shown on the OLED

The OLED's top line always shows the right address: `ONLINE 192.168.1.57` or `OFFLINE 192.168.4.1`. Some Android phones don't support `.local` names; use the IP instead.



Figure 3. Normal state. The green SAFE banner, dial inside the shaded 2–8 °C arc, and history with past excursions shaded red. *Screenshot uses simulated data from the preview tool.*



Figure 4. Excursion state. The banner turns red and pulses, the dial and background turn red, the buzzer beeps every 5 s and the LED flashes. **Silence alarm** mutes only the buzzer, for 10 minutes.

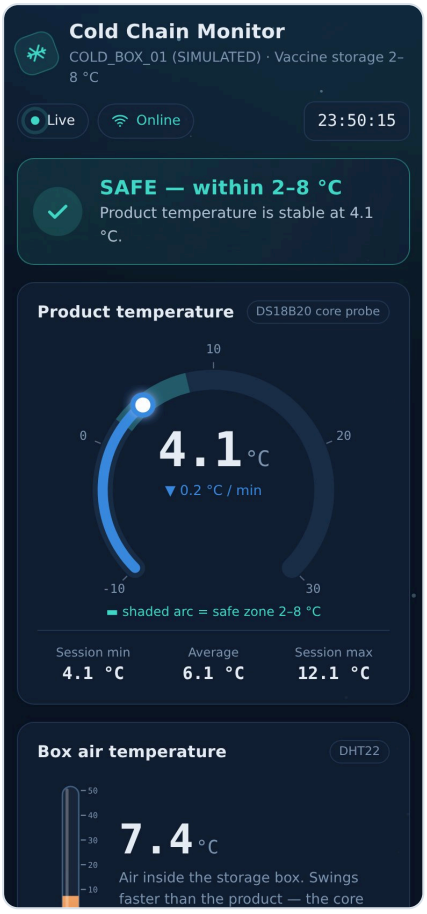


Figure 5. On a phone, the same page stacks into one column.

Reading each panel

Top chips	**Live** (pulsing) means fresh data. **Sensor stale** means no new reading for 20 s. **Connection lost** means your phone lost the unit.
Product dial	The number that decides safety. The arrow shows °C change per minute.
Box air	Changes faster than the product. Useful as an early warning (lid open).
Humidity	Above 70% the page warns about condensation on vials and labels.
Holdover	Only when the Flask server is reachable. It's an ML estimate, not a guarantee. The hourglass is greyed out when there's no prediction.
History	"Live 10 min" (5 s points) / "Trend 2 h" (30 s points). Tap or hover for exact values. "Table" gives the numbers.
Time in safe range	Green ≥ 95%, amber 80–95%, red < 80%.
SD backlog	Rows waiting to reach the Flask server. It should go back to 0 after WiFi returns.
Download	On-board log = every 30 s reading, whether uploaded or not (last column = 1 if uploaded).

7. Testing checklist

Do these in order after building. Each test proves one feature, which also works well for your project report.

#	Test	How	Expected result
1	Dashboard opens	Join the ColdChain-BOX01 WiFi and open 192.168.4.1	The splash snowflake appears, then the dashboard. The "Live" chip pulses.
2	Sensors are real	Hold the DS18B20 tip in your hand	The dial climbs towards ~30 °C within 5–10 s. "▲ x °C / min" appears.
3	Warm excursion alarm	Put the probe in warm water (> 8 °C)	Red banner "TOO WARM", buzzer beeps, LED flashes, and an EXCURSION event appears in the log.
4	Cold excursion alarm	Put the probe in ice water (< 2 °C)	"TOO COLD — freezing risk" banner.
5	Silence button	During an alarm, press "Silence alarm"	Buzzer stops, LED keeps flashing, and the button shows a countdown.
6	Recovery	Return the probe to 2–8 °C (fridge water)	Green banner; event "Back in safe range after N min".
7	Offline logging	Switch off the router for 3 minutes	WiFi tile shows "Offline", the SD backlog grows by ~6 rows, and the dashboard **still works** over the hotspot.
8	Sync on reconnect	Switch the router back on (Flask running)	Event "Backlog synced: N sent", SD backlog returns to 0, and the rows appear in Flask tagged `backlog`.
9	Reboot keeps history	Press EN/RESET after 10+ minutes	The "Trend 2 h" chart still shows the earlier data.
10	CSV download	Tap "On-board log"	A CSV opens in Excel with timestamp, temperatures, humidity, excursion and uploaded columns.

Safety note for real vaccines

This is a student prototype. Test it next to a certified thermometer or data logger for a few days before relying on it. The DS18B20 is accurate to ±0.5 °C and the DHT22 to about ±0.5 °C / ±2–5 % RH.

8. Troubleshooting

Problem	Likely cause	Fix
Page says "Dashboard files not uploaded yet"	LittleFS upload (step 6) skipped	Close the Serial Monitor → Ctrl+Shift+P → "Upload LittleFS to Pico/ESP8266/ESP32".
"Upload LittleFS" command missing	Plugin not in the right folder	The `.vsix` must be directly inside `.arduinoIDE/plugins/`. Restart the IDE fully.
LittleFS upload fails "port busy"	Serial Monitor open	Close the Serial Monitor tab and retry.
"Sketch too big"	Default partition scheme	Tools → Partition Scheme → **No OTA (2MB APP/2MB SPIFFS)**, then re-upload **both** LittleFS and the sketch.
Core temp shows −127 or banner "DS18B20 probe read error"	Missing 4.7 kΩ pull-up, or a loose yellow wire	Add the resistor between DATA (GPIO 4) and 3V3. Check the wire colours (some probes use white for DATA).
"DHT22 read error"	Wrong pin, or no pull-up on a bare sensor	DATA → GPIO 27. Add 10 kΩ to 3V3 for the 4-pin version.
Times are wrong / show 00:0x	RTC not found or battery flat	Check SDA 21 / SCL 22 and fit a CR2032. Connecting to the router fixes the time automatically (NTP).
Can't open coldchain.local	Phone doesn't support mDNS	Use the IP address shown on the OLED.
Phone keeps leaving the hotspot	Phone looks for internet	Choose "Stay connected / Use without internet" when asked, or turn off mobile data while viewing.
Hotspot drops for a moment every 15 s	Router details set, but the router is out of range	Normal while it searches (hotspot and router share one radio). Leave WIFI_SSID as the placeholder if you only use the hotspot.
Holdover shows "--"	Flask server not reachable	Run `python app.py` on the laptop, set SERVER_HOST to the laptop's IP, and put both on the same router.
SD tile "No card"	Card not FAT32, or wrong VCC	Format as FAT32 (≤ 32 GB). Power most modules from 5V.
Stuck at "Connecting…" when uploading	Board not in download mode	Hold BOOT while the upload starts. Try another USB cable (some are charge-only).

9. Files and what changed

```
coldchain_project_littlefs/
├── firmware/
│   ├── cold_chain_esp32/
│   │   ├── cold_chain_esp32.ino     ← UPDATED (v2): web server, LittleFS, hotspot, 5 s
sampling
│   │   └── data/                    ← NEW: the website stored in LittleFS
│   │       ├── index.html            page layout
│   │       ├── style.css             colours, animations, phone layout
│   │       └── app.js                live updates, gauge, charts, events
│   └── README_WIRING.md             ← UPDATED: partition scheme + LittleFS upload steps
├── tools/
│   └── mock_device.py               ← NEW: preview the dashboard on a laptop (simulated data)
├── backend/  (Flask, ML model, SQLite)   unchanged
├── wokwi/                                unchanged
└── docs/                                 unchanged
```

Firmware changes in detail

- **Web server** (`WebServer` , port 80) serves the `data/` files from LittleFS and a JSON API: `/api/live` , `/api/history?range=live|trend` , `/api/events` , `/api/mute` , plus CSV downloads at `/download/log.csv` , `/download/log_old.csv` , `/download/backlog.csv` .
- **WiFi mode AP+STA**: the hotspot is always on, the router connection is optional, and reconnection no longer freezes the loop (v1 blocked for 8 s per attempt).
- **Non-blocking DS18B20** conversion, so the web page stays responsive.
- **5 s live sampling** for the dashboard. The **30 s** log/upload cycle, JSON fields and SD CSV format are the same as v1, so the Flask backend needs no changes.
- Reads `predicted_holdover_hours` from the Flask reply and shows it on the unit.
- **LittleFS log** with automatic rotation. It is replayed at boot to refill the 2 h chart.
- **Event log** (last 24): boot, excursion start/end, WiFi lost/restored, server reachable/unreachable, backlog synced, sensor faults, alarm muted.
- **Alarm**: the LED blinks for the whole excursion. The buzzer can be muted for 10 min from the page.
- **NTP → RTC** time correction when online. The OLED shows the dashboard address.
- Uses the **ArduinoJson 7** API (`JsonDocument`).

10. What was verified, and what still needs testing

Check	Result	Details
Firmware compiles for ESP32 Dev Module	**Passed**	arduino-cli with esp32 core 3.3.10 and ArduinoJson 7.4.3: no errors or sketch warnings. 1 267 963 bytes (60%) with "No OTA"; RAM 62 KB (18%).
LittleFS image builds from `data/`	**Passed**	mklittlefs packed index.html, style.css, app.js (66 KB) into the 1.9 MB partition.
Dashboard renders and animates	**Passed**	Headless Chromium at 1440 px and 390 px (phone) widths, against the mock device: no JavaScript errors.
Interactive features	**Passed**	Chart hover tooltip, Live/Trend switch, table view (40 rows), and the Silence alarm button (countdown shown) all worked in the browser test.
Chart colours	**Passed**	Core (blue) vs air (orange) checked for colour-blind separation. Alarm states always have an icon and text, not colour alone.
Running on a real ESP32 with sensors	**Not yet tested**	No board was connected during development. Run the testing checklist in section 7 on your hardware.
Real-hardware behaviour	**Not yet tested**	Sensor timing, SD + LittleFS together, the hotspot on a live board, the Flask round-trip, and battery life all need checking on the physical unit.

> **Screenshots in this guide**
> They were taken from the preview tool (`tools/mock_device.py`) with **simulated** readings, including a staged warm-up excursion. Your unit will show your real sensor values.

Security (before any real deployment)

- Change `AP_PASSWORD` from the default `coldchain123`. Anyone who joins the hotspot can view the data and press "Silence alarm".
- The Flask `API_KEY` is still the shared default from v1. Change it in both the sketch and `backend/app.py`.
- The ESP32 page has no login. It's intended for a private hotspot or lab network, not the open internet.